GS08 · Matrículas y Notas

Informe del proyecto — reimplementación del sistema administrativo de matrículas y notas del curso Web III (PHP + MySQL) sobre FastAPI + PostgreSQL 16 + Vue 3.

Dato	Valor
Curso	GS08
Proyecto	Matrículas y Notas (MVP)
Docente	Dr. Jack Harry Peralta Cáceres
Integrantes	Arteaga Rullier, Roberto (Scrum Master + DevOps) · Díaz Cárdenas, Jorge Luis (Backend) · Inocencio Gargate, Dalcir (Frontend + Documentador)
Fecha del informe	21/09/2026
Repositorio	D:\dev\equipo\gs08-matriculas (rama main, commit e051d43 + trabajo del Sprint 1 sin commitear)
Versión	BORRADOR — la portada, el formato según las normas de la facultad y la firma final los cierra el grupo (decisión D-22)

Cómo leer este informe: cada afirmación cita el archivo o el comando del que sale y las salidas están pegadas tal como se obtuvieron. Lo que todavía no existe se escribe como «pendiente — bloqueado por T<xx>», nunca como hecho. Las fuentes completas son los documentos de docs/ (alcance, backlog, decisiones, arquitectura, manual técnico, plan de pruebas, reporte de bugs, actas, seguridad).

Índice

GS08 · Matrículas y Notas 1
Índice 1
1. Resumen ejecutivo 3
2. Problema y alcance 3
3. Arquitectura y stack 4
4. Modelo de datos y reglas de negocio 4
5. API: contrato y comportamiento 5
6. Infraestructura e integración continua 5
7. Kubernetes local y observabilidad 6
8. Calidad: criterios de aceptación y bugs 6
9. Seguridad, ética y sostenibilidad 7
10. Estado por sprint y bloqueos 7
11. Conclusiones 8
Firma grupal 9
Anexo A. Cómo se reproduce todo lo que dice este informe 10

1. Resumen ejecutivo

El proyecto reimplementa el sistema administrativo PHP + MySQL del curso Web III (4 tablas, sin API, sin pruebas, sin contenedores y sin notas) sobre un stack moderno: API FastAPI con contrato probado, PostgreSQL 16 con las reglas de negocio dentro de la base, SPA Vue 3, contenedores Docker, integración continua, Kubernetes local y observabilidad con Prometheus y Grafana.

El alcance está cerrado en 39 criterios de aceptación con identificador único (C-01 a C-39) y 20 puntos explícitamente fuera de la versión 1. El caso de uso principal —«de la matrícula a la nota»— es la prueba de aceptación y también el guion de la exposición.

Estado al 21/09/2026: el Sprint 0 está cerrado con evidencia; el API real ya existe (28 endpoints publicados) y las notas y la boleta funcionan; quedan dos bugs abiertos en los POST de matrículas y de usuarios (500) y el despliegue de Kubernetes está bloqueado por la decisión de instalar minikube.

2. Problema y alcance

El sistema legacy funciona, pero no se puede sostener: cada pantalla arma su propio SQL, los listados no paginan, las reglas viven dispersas en PHP (borrar un curso arrastra sus matrículas en silencio), el buscador usa LIKE y en PostgreSQL devuelve 0 filas sin avisar, no hay pruebas, ni contenedores, ni métricas, ni despliegue reproducible y no tiene notas aunque el proyecto se llame «Matrículas y Notas». Fuente: docs/alcance-mvp.md §1, docs/analisis/modelo-datos.md §6.

Bloque	Qué entra en el MVP	Criterios
AP-01	Autenticación con usuario o email, usuario inactivo no entra, hash $2y$ del seed, nadie desactiva su propia cuenta	C-01 a C-04
AP-02	Panel con 4 KPIs, top 5 de cursos y últimos 6 estudiantes	C-05 a C-07
AP-03	Estudiantes: buscador sin distinguir mayúsculas ni tildes, paginado, alta/edición, baja lógica	C-08 a C-12
AP-04	Cursos: créditos 1-10, horas, código único, baja lógica	C-13 a C-15
AP-05	Matrículas: alta, duplicado 409, periodo 422, retirar sin borrar	C-16 a C-21
AP-06	Usuarios: solo admin, roles, 403 por rol	C-22 y C-23
AP-07	Notas y boleta: nota 0-20, media por curso y promedio ponderado por créditos	C-24 a C-29
AP-08	Borrados destructivos: 409 con el conteo y doble confirmación	C-30 a C-32
AP-09	Transversal: contenedores, balanceo y failover, Grafana, Kubernetes, CI y pruebas	C-33 a C-39

Fuera de la versión 1 (20 puntos, docs/alcance-mvp.md §5): portal del estudiante y rol docente, pagos y caja, horarios y cupos, asistencia, actas y certificados, historial multi-periodo, pesos por tipo de evaluación, ETL del legacy, auditoría, recuperación de clave y 2FA, notificaciones, app móvil, multi-tenant, HPA y TLS, nube, backups, pruebas de carga, exportación a Excel/PDF y Pinia.

3. Arquitectura y stack

La arquitectura se documenta con cinco diagramas (componentes, flujo de una petición con balanceo y failover, modelo de datos, despliegue en Kubernetes y observabilidad) en docs/arquitectura.md, y los diagramas fueron verificados con un analizador de Mermaid: los 5 parsean sin error.

Capa	Tecnología y versión real	Fuente
API	FastAPI 0.141.1 + uvicorn 0.53.0 sobre Python 3.12.14	docs/devops/plan-contenedores-ci.md §3
Base de datos	PostgreSQL 16.15 (imagen postgres:16.15-alpine3.24)	docker-compose.yml
Frontend	Vue 3 + Vite + Tailwind sobre Node 22.23.2 y nginx 1.31.6	frontend/Dockerfile
Contenedores	Docker 29.8.0 + Compose v5.5.1, red gs08-net	docker-compose.yml
Observabilidad	Prometheus v3.13.3 (scrape 15 s) + Grafana 12.4.11	infra/prometheus/prometheus.yml
Kubernetes	kubectl v1.36.1 + kustomize v5.8.1, 11 objetos en k8s/	kubectl kustomize k8s/
CI/CD	GitHub Actions (CI en PR y push a main; CD a GHCR con SBOM y provenance)	.github/workflows/

Decisión de arquitectura central: se reimplementa el legacy, no se migra (D-00). Del sistema original se conservan el modelo de datos y la experiencia de usuario, no el código.

4. Modelo de datos y reglas de negocio

El esquema se tradujo de MySQL 8 a PostgreSQL 16 y se verificó corriendo: 4 tablas con 16 CHECK, 6 UNIQUE, 2 FK y 7 índices, más la vista v_matriculas_detalle; el bloque de notas (tabla notas y vista v_notas_detalle) se sumó después como esquema aditivo (D-01).

Objeto	Contenido con el seed	Script
usuarios	1 usuario admin (admin / Admin123!, hash bcrypt $2y$ coste 10)	db/init/03-seed.sql
estudiantes	12 estudiantes, DNI de 45123456 a 56234567	db/init/03-seed.sql
cursos	7 cursos (C101 a C205), créditos 2 a 4	db/init/03-seed.sql
matriculas	24 matrículas: 23 activas + 1 retirada (id 13, estudiante 7, curso 4)	db/init/03-seed.sql
v_matriculas_detalle	matrículas unidas con estudiante y curso (24 filas)	db/init/02-view.sql
notas	tabla de notas con tipo	db/init/04-notas.sql

	practica/parcial/final, numero >= 1 y nota 0-20	

Las reglas de negocio RN-01 a RN-16 se extrajeron de los módulos PHP del legacy y viven en la base (CHECK, UNIQUE y FK), con la API repitiendo la validación para poder responder 422 con mensaje. Verificación: 54 comprobaciones del modelo y 32 de notas, incluidas las pruebas negativas que exigen que la base rechace cada regla violada, con 0 fallas y salida real en db/verificacion/.

Hallazgo que cambió el diseño del buscador: el LIKE del legacy devuelve 0 filas en PostgreSQL (medido) y con ILIKE sin la extensión unaccent tampoco encuentra «Huamán» buscando «huaman». El buscador usa ILIKE con unaccent (D-11) y el API real ya responde 1 fila en las tres variantes.

5. API: contrato y comportamiento

El API publica 28 endpoints documentados en su propio OpenAPI (título «GS08 · Matrículas y Notas», versión 0.1.0). El contrato completo, con el estado real de cada fila (implementado y verificado, con 500 hoy, o escrito sin verificar), está en docs/manual-tecnico.md §2.2.

Grupo	Endpoints	Comportamiento verificado
Salud	/health, /api/v1/health, /metrics	200 y texto Prometheus (métricas directas, D-17)
Autenticación	POST /api/v1/auth/login, GET /api/v1/auth/yo	200 con token; 401 con clave mala o usuario inactivo
Panel	GET /api/v1/dashboard	12 estudiantes, 7 cursos, 23 matrículas activas, 1 usuario
Estudiantes	GET/POST/PUT/DELETE	buscador 1/1/1 con y sin tildes; 422 con el campo; 409 al borrar con matrículas
Cursos	GET/POST/PUT/DELETE	422 en créditos 0 u 11 y horas 0; 409 en código repetido
Matrículas	GET/POST/PUT/DELETE	listado y retirar OK; POST responde 500 (BUG-08, en arreglo)
Usuarios	GET/POST/PUT/DELETE	POST responde 500 (BUG-09, en arreglo); 403 por rol pendiente de esa corrección
Notas y boleta	POST /matriculas/{id}/notas, PUT/DELETE /notas/{id}, GET /estudiantes/{id}/boleta	201/409/422 correctos; boleta del estudiante 1: curso 16.00 y promedio ponderado 13.43

Formato de error uniforme {"detail": "…"} con el mensaje del backend, que el SPA muestra en pantalla. Regla transversal: ningún dato del usuario puede provocar un 500; los rechazos son 401, 403, 404, 409 o 422.

6. Infraestructura e integración continua

Elemento	Realidad verificada
Contenedores	6 en total con el perfil obs: web (8080), api (8000), api-b, db (5432), prometheus (9090), grafana (3000)
Balanceo	12 peticiones seguidas alternaron las dos instancias del API (X-Upstream-Addr)

Failover	con gs08-api-b apagado las 6 peticiones siguieron dando 200 con reintento en la otra instancia
Verificación del stack	python scripts/verificar_stack.py → 9 comprobaciones OK, 0 fallas, exit 0
Validación sin Docker	python scripts/validar_infra.py → 21 comprobaciones OK, 0 fallas (es lo que corre el CI en cada pull request)
CI	5 jobs: infra, backend (ruff + pytest con cobertura mínima 60 %), frontend (eslint + vitest + build), imágenes (build + smoke + trivy) y la puerta ci-ok
CD	publica en GHCR con SBOM y provenance tras un CI verde en main; el despliegue a Kubernetes se activa con DEPLOY_ENABLED=true

Honestidad sobre el estado: el CI y el CD están escritos y validados offline, pero todavía no han corrido nunca en GitHub porque el repositorio no tiene remoto configurado (T4.1). El compose del repositorio sigue sirviendo el arnés de humo con el que se validó la infraestructura antes de que existiera el código; integrar el API real al compose es T2.6.

7. Kubernetes local y observabilidad

Los manifiestos están escritos y renderizan 11 objetos (Namespace, ConfigMap, PVC, Job de migraciones, los Deployment y Service de db, api y web, y el Ingress). La validación offline es kubectl kustomize k8s/ porque kubectl apply --dry-run=client no puede validar sin cluster.

Estado real, tal como se va a exponer: el cluster NO está desplegado en la máquina de desarrollo. kubectl config current-context responde «error: current-context is not set» y minikube, kind ni k3s están instalados; instalar minikube es software nuevo en la máquina y espera la aprobación del dueño (A-02). No se reporta kubectl get pods hasta que exista un cluster.

La observabilidad sí corre: 3 targets en estado up, un dashboard API RED con 9 paneles con datos (método Rate-Errors-Duration), provisionado desde el archivo del repositorio. Limitación declarada: PostgreSQL y nginx no exponen métricas Prometheus sin un exporter propio.

8. Calidad: criterios de aceptación y bugs

Los 39 criterios tienen matriz de trazabilidad con su caso de prueba y su estado real (docs/qa/plan-pruebas.md §3). El estado al 21/09/2026, con el API real:

Estado	Criterios	Detalle
Verificados con corrida real	22	C-01, C-04 a C-11, C-13, C-14, C-18, C-19, C-24 a C-32 y C-35
Incumplidos por bugs abiertos	4	C-16, C-17, C-22 y C-23 (los POST de matrículas y usuarios dan 500)
En arreglo con decisión tomada	2	C-33 (TC-01) y C-34 (TC-03), a cargo de DevOps y verificados por QA
Pendientes del guion de	6	necesitan el SPA (T2.4)

interfaz		
Bloqueados con motivo	5	pytest, cluster (A-02), CI sin remoto y SPA

Probar encontró 9 bugs en total y 4 bugs en el propio script de pruebas. Los hallazgos incluyen un verificador de infraestructura que fallaba en un entorno recién levantado, un criterio que citaba un correo y un DNI que no existían en los datos, un /metrics que devolvía el index.html con 200, cabeceras de seguridad que no llegaban al API, la falta de la extensión unaccent y los dos 500 de esta ronda. El reporte completo, con pasos, esperado, obtenido, evidencia y estado, está en docs/qa/reporte-bugs.md.

Regla de la casa: un bug no se cierra por decisión, se cierra con la corrida verde pegada. Por eso C-33 y C-34 figuran como «en arreglo» y no como cumplidos.

9. Seguridad, ética y sostenibilidad

- Seguridad: bcrypt coste 10 y verificación del hash $2y$ del seed con la librería del API; roles admin y asistente con 403 por rol; 403 y 409 como contrato; 4 cabeceras de seguridad en nginx; ningún secreto en el repositorio (plantilla de Secret y .env ignorados); SBOM y provenance en GHCR; contenedores sin privilegios.
- Superficie expuesta: 5 puertos publicados y todos solo en local; el puerto de PostgreSQL no se publica en el cluster. Las métricas se leen directo del API y nginx responde 404 en /health y /metrics para que un chequeo de estado no dé por bueno un endpoint vacío.
- Ética y datos personales: DNI, nombres, contacto y notas son datos personales (Ley 29733); el sistema aplica minimización (no guarda datos de salud, de menores ni campos «por si acaso») y trabaja con los datos ficticios del legacy. El mapa legal a verificar con asesoría está en docs/seguridad-etica-sostenibilidad.md §5.
- Sostenibilidad: costo S/ 0 (todo open source, sin nube y con CI en el plan gratuito), imágenes alpine y multi-etapa, perfiles de compose, retención acotada de métricas y logs, y mantenibilidad medida (54 + 32 + 9 + 21 comprobaciones, cada una con un comando).
- Uso de IA declarado: el proyecto se construyó con un equipo de agentes de IA bajo la dirección y validación del dueño; ninguna evidencia es generada, todas las salidas provienen de comandos reales, y los números se verificaron a mano por el propio equipo.

10. Estado por sprint y bloqueos

Sprint	Alcance	Estado
Sprint 0	alcance, modelo de datos, contenedores, CI, Kubernetes, plan de pruebas	Cerrado con evidencia
Sprint 1	API núcleo y cluster local	En curso: API con 28 endpoints y 22 criterios verdes; dos 500 abiertos y cluster sin desplegar
Sprint 2	matrículas, usuarios y SPA	Endpoints escritos; el SPA no existe (T2.4)
Sprint 3	notas y boleta	Funcionando y verificado a mano; falta el recorrido completo, que hoy se corta en matrículas

Sprint 4	cierre, Kubernetes real y entregables	Pendiente

Bloqueos declarados: Kubernetes real (falta la aprobación para instalar minikube, A-02), repositorio público y CI sin ejecutar (T4.1), SPA (T2.4) y el recorrido de extremo a extremo (BUG-08). La fecha de entrega y de exposición sigue pendiente de confirmación (A-01).

Orden de corte si el tiempo aprieta, fijado de antemano y comunicado en la exposición: primero se corta el bloque de notas, después la regresión sobre Kubernetes; no se corta Kubernetes como entregable, ni la observabilidad, ni la tabla de criterios con evidencia.

11. Conclusiones

- El MVP resuelve el mismo problema administrativo del legacy con ingeniería verificable: contrato de API probado, reglas dentro de la base, contenedores, balanceo con failover, observabilidad y despliegue reproducible.
- El valor del método está en la evidencia: cada cifra de este informe sale de un comando corrido, y lo que no se pudo probar está declarado como bloqueado.
- Probar temprano pagó: los defectos de infraestructura se corrigieron antes de que existiera el código de la aplicación, y los hallazgos de los criterios se convirtieron en decisiones y tareas con dueño.
- Lo que sigue: cerrar los dos 500 de los POST (BUG-08 y BUG-09), completar la extensión unaccent en la migración de Alembic, desplegar el cluster cuando se apruebe minikube, construir el SPA y correr el recorrido completo de la matrícula a la nota.

Firma grupal

El presente informe corresponde al trabajo del grupo y se presenta con la firma de los tres integrantes.

Integrante	Rol en el proyecto	Firma	Fecha
Arteaga Rullier, Roberto	Scrum Master + DevOps		
Díaz Cárdenas, Jorge Luis	Backend		
Inocencio Gargate, Dalcir	Frontend + Documentador		

Docente del curso: Dr. Jack Harry Peralta Cáceres

Anexo A. Cómo se reproduce todo lo que dice este informe

Todos los comandos se ejecutan desde D:\dev\equipo\gs08-matriculas.

Qué comprueba	Comando	Resultado real
Infraestructura sin Docker	python scripts/validar_infra.py	21 comprobaciones OK, 0 fallas
Stack en marcha	python scripts/verificar_stack.py	9 comprobaciones OK, 0 fallas (exit 0)
Humo por el proxy	bash scripts/smoke_api.sh --infra	8 OK, 0 fallas, exit 0
Modelo de datos	docker exec -i gs08-analista-verif psql -q -U gs08 -d gs08_matriculas < db/verificacion/verificar_modelo.sql	54 comprobaciones OK, 0 fallas
Notas y boleta en la base	… < db/verificacion/verificar_notas.sql	32 comprobaciones OK, 0 fallas
Manifiestos de Kubernetes	kubectl kustomize k8s/	11 objetos
Contrato de la API	bash scripts/smoke_api.sh --solo-contrato --e2e --url http://localhost:8010	matriz de los 39 criterios con su estado

Documentos fuente: docs/alcance-mvp.md · docs/backlog-sprints.md · docs/decisiones.md · docs/arquitectura.md · docs/manual-tecnico.md · docs/seguridad-etica-sostenibilidad.md · docs/analisis/modelo-datos.md · docs/analisis/datos-seed.md · docs/devops/evidencia-sprint0.md · docs/devops/plan-contenedores-ci.md · docs/qa/plan-pruebas.md · docs/qa/reporte-bugs.md · docs/actas/acta-sprint-0.md · docs/actas/retrospectiva-sprint-0.md.